Habit Tracker: Habit Lifecycle Application

"A Python system that tracks habits, monitors consistency, and analyzes behavioral patterns."

Development Phase Presentation

March 09, 2026

Fatima Ezzahrae Ezzouzi

Tutor: **Mirmehdi Seyedebrahimi**

Bachelor: Applied AI

Course: Object-Oriented Functional Programming with Python

Development Phase Roadmap

1. Project Overview
2. System Architecture
3. Habit Class & Event Log
4. Storage Approach (SQLite Database) & Data Flow
5. Live Demo - Full CLI Workflow
6. Analytics Functions (Functional Style)
7. Predefined Habits & Testing Strategy
8. Progress vs Original plan & Next Steps

Project Overview

Goal

The goal of the application is to **help users build and maintain consistent habits by making habit tracking simple and effective**.
It aims to **provide clear insights into user progress**, helping users stay motivated and improve their routines over time.

Core Features

- Create & edit habits & delete habits
- Log daily/weekly completion logging
- Streak & broken-habit analysis
- Visualize progress through charts and export habit data to HTML reports.
- All habit data and completion history are stored persistently in an SQLite database.

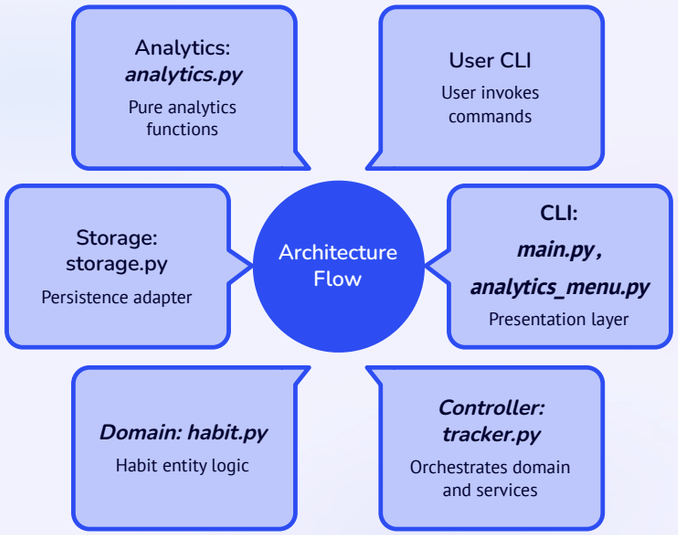
Phase 1 Summary

The conception phase defined the **system architecture and structural design** of the application.
A **modular architecture** was established, consisting of the Habit domain model, HabitTracker controller, analytics module, and SQLite storage layer.
Each module has a **clear responsibility**, separating business logic, data storage, and user interaction.
This structure ensures the application is **maintainable, scalable, and testable**, supported by automated testing with **pytest.**

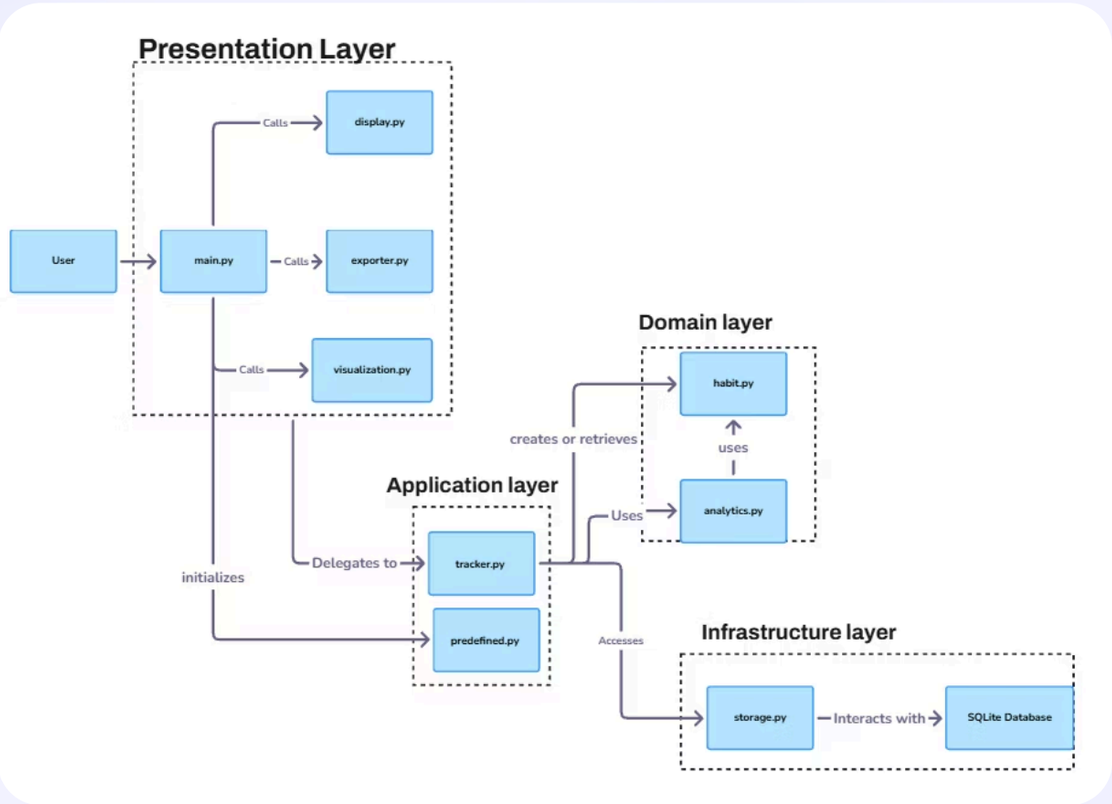
System Architecture

Architecture

- The system follows a **layered architecture** separating presentation, application, domain, and infrastructure concerns.



- The **Presentation Layer** handles user interaction through the CLI and visualization/export modules.
- The **Application Layer** (tracker.py) orchestrates operations and coordinates between the UI and business logic.
- The **Domain Layer** contains the core business logic (habit.py) and pure analytics functions (analytics.py).
- The **Infrastructure Layer** manages persistence using SQLite through storage.py.



Habit Class & Event Log

Habit class

- The **Habit class** is the core domain model that represents a single habit and stores its main attributes: *name, category, frequency (daily/weekly), duration, creation date,* and a list of **completion timestamps**(event log).
- Each time a habit is completed, the method `add_completion()` records the event in the **event log (completions list)**, which acts as the source of truth for tracking progress.
- The class dynamically calculates **current streaks, longest streaks, and broken habits** from this event log instead of storing derived values.
- Utility methods such as `to_dict()` allow the habit data to be easily used by analytics, visualization, and export modules.

Design decision

Store **raw completion events (timestamps)** instead of pre-computed streaks → all analytics (current streak, longest streak, broken habits) are dynamically calculated from these raw events, ensuring **accuracy, flexibility, and consistency**.

Computed metrics

- current streak
- longest streak
- detect broken habits
- The **Habit class** represents a single habit in the system and manages its lifecycle.

This approach keeps the data model simple and ensures analytics are always calculated from real completion events.

Event Log Example

Habit: **Drink Water (Daily)**

From this event log the system computes:

Current Streak: 2 days
Longest Streak: 3 days
Broken Habit: Yes (missed Feb 4)

Storage Approach (SQLite Database) & Data Flow

Storage Approach

The application uses **SQLite** as the persistence layer to store habits and completion events.

Two tables are used:

- **habits** – stores habit definitions
- **completions** – stores timestamped habit completions

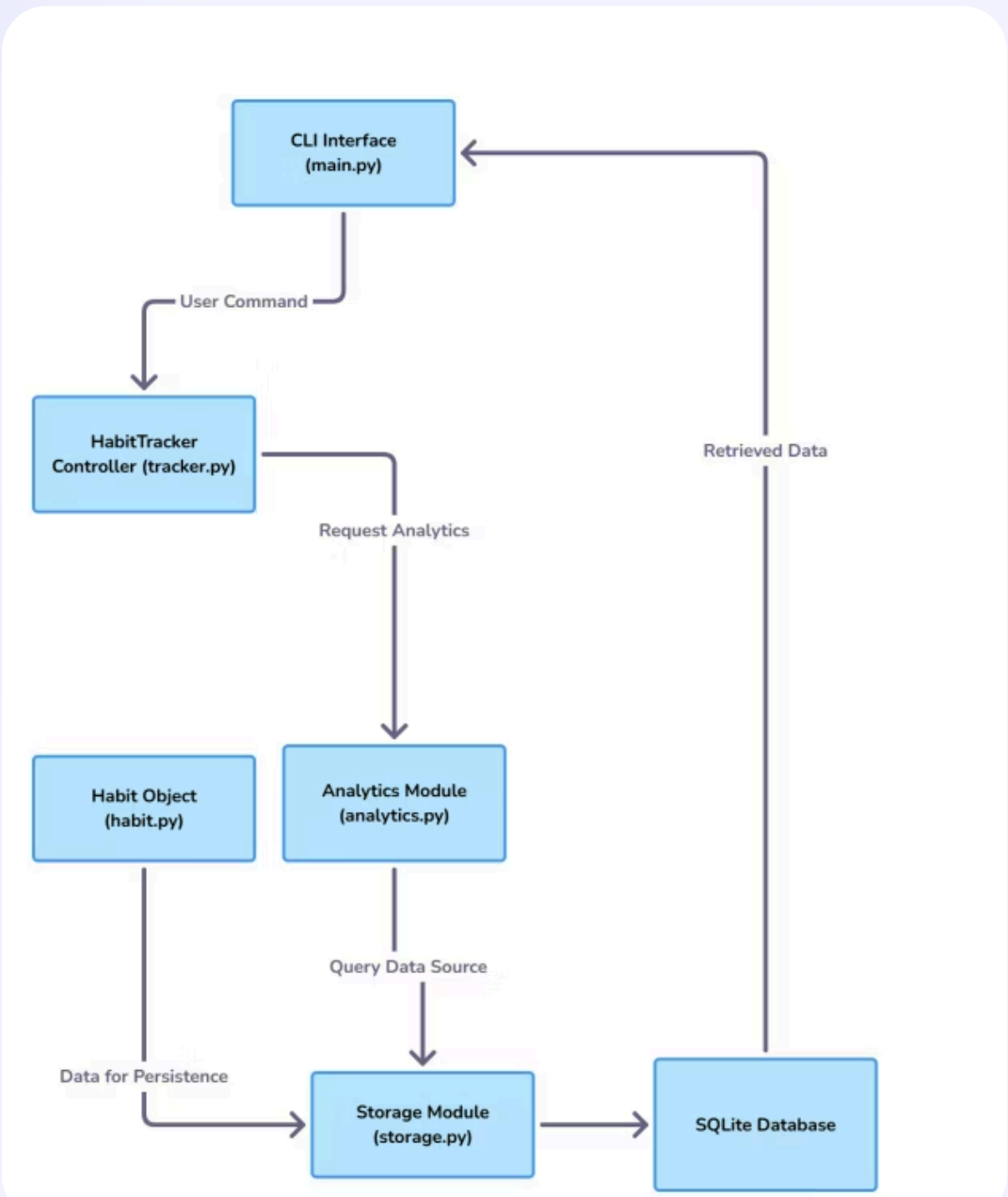
This design records **raw completion events**, allowing analytics functions to compute streaks dynamically.

Database schema (SQLite)

habits	completions
id name category frequency created_at	id habit_id completed_at

Habit Tracker Data Flow

1. User interacts with the system through the **CLI interface**.
2. The **HabitTracker controller** manages all operations.
3. Habit objects store completion events and streak logic.
4. The **Storage module** persists data in an SQLite database.
5. Analytics functions process stored data to compute habit statistics.
6. Results are displayed in the CLI, visualized in charts, or exported as HTML reports.

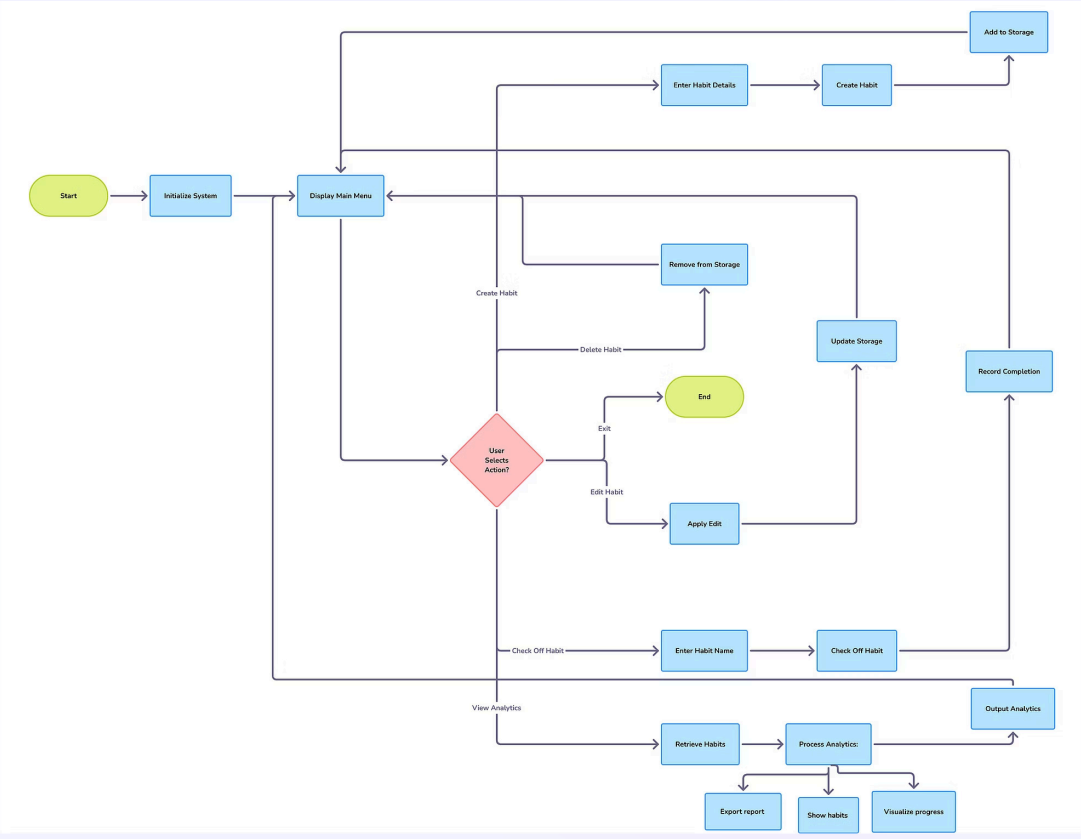


This architecture separates **data persistence, business logic, and analytics**, improving maintainability and enabling independent testing of each module.

Live Demo - Full CLI Workflow

Diagram

This diagram summarizes the main workflow of the Habit Tracker CLI, showing how user actions are processed through the tracker logic, analytics modules, and storage system.



Main Menu Interaction

Watch the Habit Tracker in action as we walk through key functionalities.

run python main.py in terminal

Loading habits from database ...

✓ Database initialized with 5 predefined habits
✓ Ready to track!

```
=== 🌱 Daily Check-in Dashboard 🌱 ===
➕ 1. Create a new habit
✅ 2. Check off a habit
📊 3. Analytics
✏️ 4. Edit habit
🗑️ 5. Delete habit
🚪 0. Exit
Choose an option:
```

when you choose ***1 : Creat habit*** , you should enter thabit details like here for (drawing)

```
hoose an option: 1
Habit name: drawing
Category: art
Frequency (daily/weekly): weekly
Duration (in days): 30
```

when you save, you see :

```
✅ Habit "drawing" created successfully!
🗓️ Scheduled for: 30 days
🎯 Frequency: Weekly
```

when u choose 2 : ***check off***

```
Choose an option: 2
Habit name to check off: drawing
```

then you will see :

```
✅ Check Off Habit
──────────────────────────────────────

Habit name: drawing

⏰ Timestamp: 2026-01-22 01:09:08

🎉 Habit "drawing" checked off!
Keep going 💪

Current Streak: 1 day
```

when you choose 3 : ***View analytics***

```
Choose an option: 3

        📊 Analytics 📊
📋 1. Show all habits (Compact view)
📊 2. Export habits to HTML table
📈 3. Habit Progress Over Time
💔 4. Show broken habits
🔥 5. Show current streak for all habits
🎳 6. Show habits by frequency
🏆 7. Show longest streak (all)
🎯 8. Show longest streak (specific)
↩️ 0. Back
👉 Choose:
```

for choice 3 - 1: ***compact view of habits***

```
ose are all habits :

📋 YOUR HABITS:

#   Name          Category    Freq   Days  Current  Best
=============================================================
1   Drink Water   Health      D      30    0        28
2   Read Book     Mind        D      90    0        20
3   Morning Walk  Fitness     D      60    0        11
4   Gym           Fitness     W      60    4        2
5   Call Family   Social      W      90    2        1
6   dancing       sport       W      90    1        1
7   writing       litterature D      30    0        0
8   drawing       art         W      30    1        1

ress Enter to continue...
```

for 3-2 : ***Habits report table***

📊 Habit Tracker Report

Generated on: January 21, 2026 at 21:04

Name	Category	Frequency	Duration	Start Date	Marked off	Last Completed	Current Streak	Longest Streak
Drink Water	Health	Daily	30	2026/01/21	79	2026/01/21	**28**	**28**
Read Book	Mind	Daily	90	2026/01/21	89	2026/01/21	**20**	**20**
Morning Walk	Fitness	Daily	60	2026/01/21	24	2026/01/21	**11**	**11**
Gym	Fitness	Weekly	60	2026/01/21	14	2026/01/21	**4**	**2**
Call Family	Social	Weekly	90	2026/01/21	13	2026/01/21	**2**	**1**
dancing	sport	Weekly	90	2026/01/21	1	2026/01/21	**1**	**1**

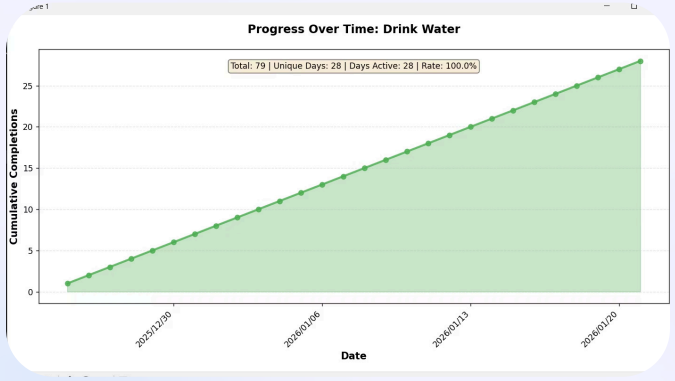
for 3-3: select habit you want to see its progress over time :

```
👉 Choose: 3

Available habits:
1. Drink Water
2. Read Book
3. Morning Walk
4. Gym
5. Call Family
6. dancing
7. writing
8. drawing

Enter habit number or name: Drink Water
```

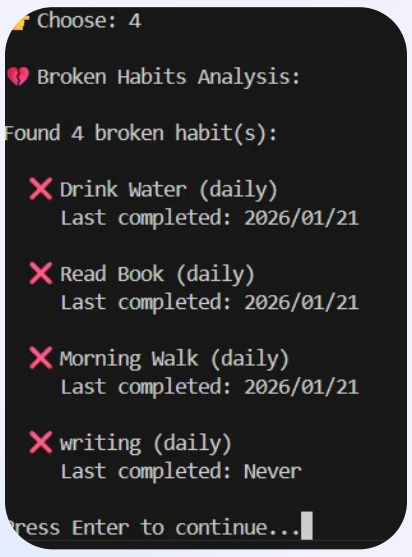
when you choose the name, the app generates for you a chart and saved it as png image .

```
Enter habit number or name: Drink Water

Generating progress chart for 'Drink Water'...

✓ Progress chart saved as 'Drink_Water_progress.png'
```

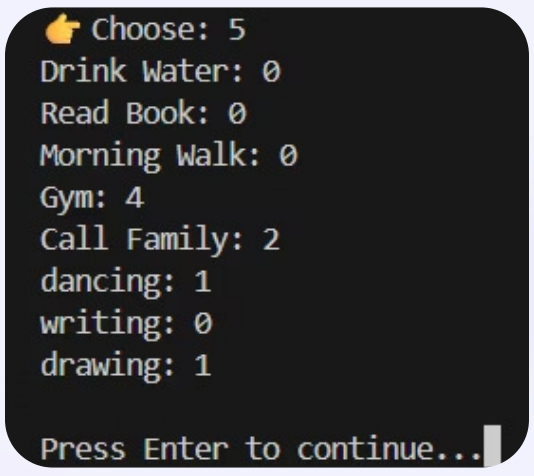
Drink Water case:



for 3-4: ***broken habits***



for 3-5 to 3-8: streaks analysis , when u choose 3. 5 , the ***current streaks*** will be shown :



Start tracking today and achieve your personal growth goals.

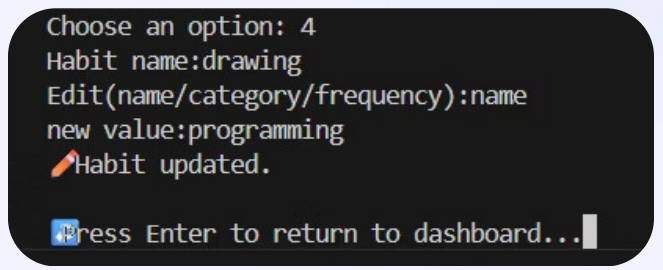
Habit Editing Process

step 1 : you choose 4 ,

step2 : you select the name ,

step 3 : you select what you wanna edit

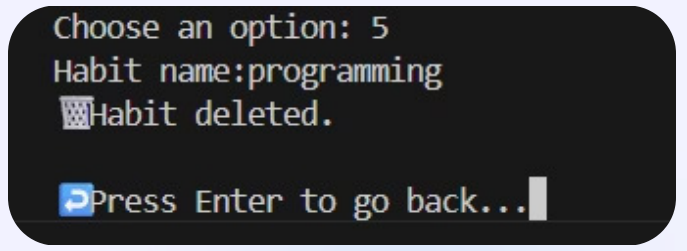
step 4 : you give a new value



Habit deleting Process

step 1 : you choose 5

step2 : you select the name of the habit ,



Analytics Functions (Functional Style)

Design

The analytics module analyzes habit performance using **functional programming principles**.
Instead of embedding analytics inside the habit class or database layer, the system uses **independent functions that operate on collections of habit objects**.

This approach improves:

- modularity
- testability
- reusability of analytics logic.

The analytics functions process **lists of Habit objects** and compute behavioral insights such as streaks and broken habits.

Implemented Analytics Functions

The current system includes the following functions:

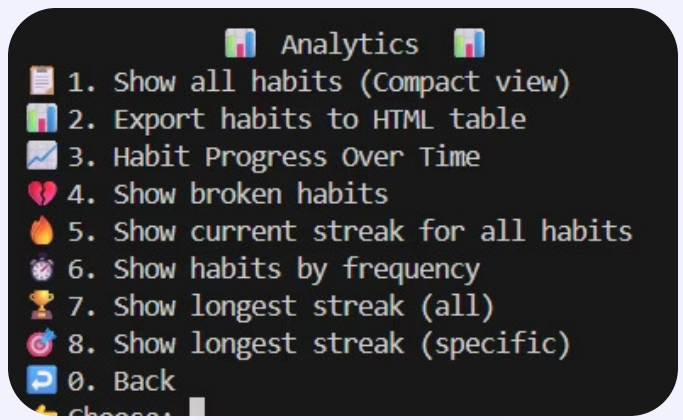
- **get_all_habits()** – returns all tracked habits
- **get_broken_habits()** – identifies habits where the streak was broken
- **get_habits_by_periodicity()** – filters habits by frequency (daily or weekly)
- **get_longest_streak_all()** – finds the longest streak across all habits
- **get_longest_streak_for_habit()** – calculates the longest streak for a specific habit.

Functional Processing Flow

Implementation Status

- analytics module implemented
- functions integrated with CLI dashboard
- tested using predefined habit dataset

These functions are used in the **Analytics menu** to provide behavioral insights.



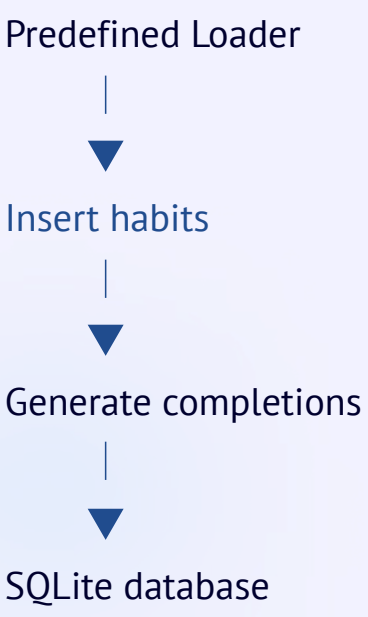
Predefined Habits & Testing Strategy

Predefined dataset (4 weeks)

- Daily: Drink Water · Read Book · Morning Walk
- Weekly: Gym · Call Family
- Simulated completions to validate streak logic

id	name	category	frequency	duration	created_at
Filter	Filter	Filter	Filter	Filter	Filter
1	Drink Water	Health	daily	30	2026-01-21T17:18:06.111471
2	Read Book	Mind	daily	90	2026-01-21T17:18:06.111507
3	Morning Walk	Fitness	daily	60	2026-01-21T17:18:06.111509
4	Gym	Fitness	weekly	60	2026-01-21T17:18:06.111511
5	Call Family	Social	weekly	90	2026-01-21T17:18:06.11151?

Testing data Diagram:



4-Week Tracking Simulation

The system generates **28 days of historical completion data**.

The dataset intentionally includes:

- Completed days
- Missed days
- Streaks and broken streaks

This timeline shows four weeks of simulated habit tracking, including completed days and missed days to demonstrate streak calculations and broken streak detection.

Week 1 : ✔️ ✔️ ✔️ ☒ ✔️ ✔️ ✔️

Week 2: ✔️ ✔️ ☒ ✔️ ✔️ ✔️ ✔️

Week 3: ✔️ ✔️ ✔️ ✔️ ✔️ ☒ ✔️

Week 4: ✔️ ✔️ ✔️ ✔️ ✔️ ✔️ ✔️

**✔️ = Habit completed

☒ = Missed day (breaks streak)

Purpose

This predefined dataset allows:

- Immediate **analytics demonstration**
- Testing **streak calculations**
- Detecting **broken habits**
- Validating the **habit tracking logic**

Testing(using pytest)

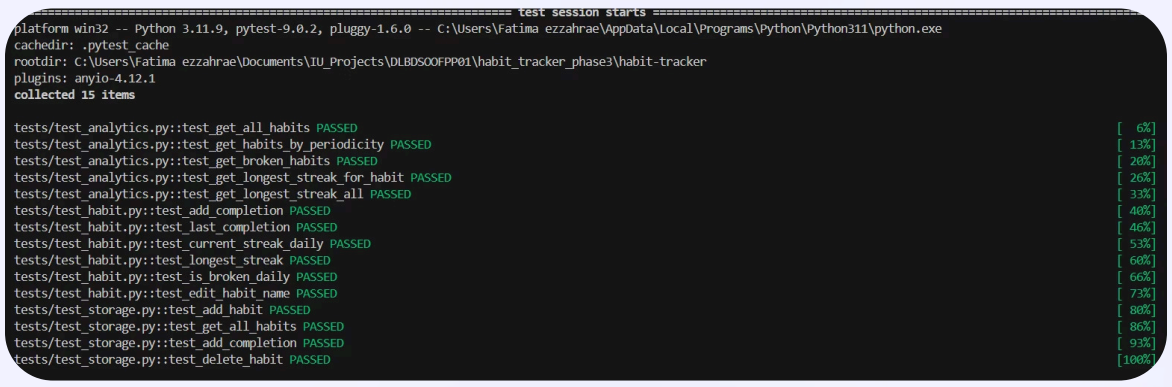
Testing Framework

- Unit tests implemented using **pytest**
- Tests validate **core logic independently from the CLI**
- Dummy habit data is used to simulate completions and streaks

Testing Areas

Component	What is Tested
Habit class	Streak calculation, completion logging
Analytics module	Filtering habits, longest streak functions
Storage layer	Database insert, retrieval, completion storage

Early Test Results



The core logic of the system is tested through unit tests for the Habit class, analytics functions, and storage layer. The tracker controller mainly orchestrates these components, so additional workflow tests may be added in the next phase.

Progress vs Original plan & Next Steps

Completed / Progress

- Modular architecture
- Change structure (clean main.py and separate some files to be organized)
- Habit class implemented
- SQLite persistence
- Analytics module and CLI
- Predefined dataset & tests
- Modifications in testing

Original plan

- Project structure unclean little bit (main page included a lot of displaying and viusalizing , exporting), now it changed to be separated

Next steps for the phase 3

- Enhance CLI
- Optimize analytics performance
- improve reporting / export features
- Improve Testing
- reviewing the code logic

Conclusion: Deliver a robust habit-tracking system that records raw events and provides behavioral analytics for sustained habit formation.